Department of

Artificial Intelligence and Machine Learning

B.Tech III Semester

Academic Year 2026-2027

Data Base Management Systems (2501IT05)

Capstone Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Artificial Intelligence and Machine Learning

HOSPITAL MANAGEMENT DATABASE SYSTEM

A Cornerstone Project Report

BATCH NUMBER : 3

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

A MANI KRISHNAN - AIML	**(25B11AI006)**
MBV JASWANTH - AIML	**(26B61AI002)**
CH VENKATA KOUSHIK - AIML	**(25B11AI218)**
B SRI RAM KISHAN - AIML	**(25B11AI116)**

Under the guidance of

Dr. P. Kishorekumar, PhD.
Associate Professor, Department of AIML

ADITYA UNIVERSITY, SURAMPALEM

Academic Year: 2026 - 2027

ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the Cornerstone project work entitled ***"HOSPITAL MANAGEMENT DATABASE SYSTEM"*** is submitted by

A MANI KRISHNAN - AIML	**(25B11AI006)**
MBV JASWANTH - AIML	**(26B61AI002)**
CH VENKATA KOUSHIK - AIML	**(25B11AI218)**
B SRI RAM KISHAN - AIML	**(25B11AI116)**

In partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2026-2027.

Project Guide

Dr. P. Kishorekumar, PhD.

Associate Professor, Department of AIML

Head of the Department:

Dr. Kovvuri N Bhargavi

Associate Professor & HOD

DECLARATION

We hereby declare that the Cornerstone project entitled **"HOSPITAL MANAGEMENT DATABASE SYSTEM"** is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. Degree in Artificial Intelligence and Machine Learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date:

By

A MANI KRISHNAN - AIML	**(25B11AI006)**
MBV JASWANTH - AIML	**(26B61AI002)**
CH VENKATA KOUSHIK - AIML	**(25B11AI218)**
B SRI RAM KISHAN - AIML	**(25B11AI116)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. P. Kishorekumar, PhD. Associate Professor, Department of AIML,** who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. B. Manikyala Rao, Associate Professor**, **Dr. R. Suneetha, Assistant Professor and Dr. D. Veeraswamy, Assistant Professor** for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. K. Naga Bhargavi,** Associate Professor & HOD**,** for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

By

A MANI KRISHNAN - AIML	**(25B11AI006)**
MBV JASWANTH - AIML	**(26B61AI002)**
CH VENKATA KOUSHIK - AIML	**(25B11AI218)**
B SRI RAM KISHAN - AIML	**(25B11AI116)**

TABLE OF CONTENTS

1. Cover Page 1
2. Certificate 2
3. Declaration 3
4. Acknowledgement 4
5. Abstract 6
6. List of Figures / List of Tables / Acronyms 7
7. Introduction 9
8. System Analysis 11
9. Database Design 13
10. ER Diagram 15
11. Relational Schema 16
12. Table Design 17
13. Normalization 21
14. SQL Implementation 23
15. SQL Queries & Screenshots 28
16. Conclusion & Future Enhancements 33
17. References 35

ABSTRACT

The Hospital Management Database System is designed to organise and manage the large volume of interconnected information that a hospital handles on a daily basis. Hospitals routinely deal with data relating to patients, doctors, departments, appointments, admissions, treatments, prescriptions, medicines, rooms, and billing. Maintaining such records manually or across disconnected spreadsheets leads to duplication, inconsistency, and difficulty in retrieving information. The proposed system models the major hospital entities and the relationships between them using a relational database approach, and stores the information in a centralized database that enforces data integrity through Primary Keys and Foreign Keys.

The database has been designed with ten major entities — Department, Patient, Medicine, Room, Doctor, Appointment, Admission, Treatment, Prescription, and Billing — connected through well-defined one-to-many relationships. The design has been normalized up to the Third Normal Form (3NF) to minimise redundancy and prevent update anomalies. The system has been implemented using SQL, covering Data Definition Language (DDL) for table creation, Data Manipulation Language (DML) for sample records, and Data Query Language (DQL) for retrieval, joins, aggregation, subqueries, and a database view. The resulting design provides a structured foundation for patient management, doctor and department management, appointment scheduling, admission and room allocation, treatment and prescription tracking, and billing management.

LIST OF FIGURES

Figure 1. ER Diagram of the Hospital Management Database System

LIST OF TABLES

Table 1. Major Database Entities

Table 2. Entity Relationships and Cardinality

Table 3. Relational Schema Summary

Table 4. Department Table Structure

Table 5. Patient Table Structure

Table 6. Medicine Table Structure

Table 7. Room Table Structure

Table 8. Doctor Table Structure

Table 9. Appointment Table Structure

Table 10. Admission Table Structure

Table 11. Treatment Table Structure

Table 12. Prescription Table Structure

Table 13. Billing Table Structure

Table 14. Keys and Constraints Summary

ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form

1. INTRODUCTION

1.1 Brief Information about the Project

Hospitals handle large amounts of information related to patients, doctors, departments, appointments, admissions, treatments, prescriptions, medicines, rooms, and billing. A database provides a structured and efficient way to organise and manage these interconnected hospital records. The proposed Hospital Management System models the major hospital entities and their relationships using a relational database approach. The project focuses on designing the database structure — including entities, attributes, Primary Keys, Foreign Keys, relationships, and the ER diagram — and implementing it through SQL.

1.2 Motivation and Contribution of the Project

The project is motivated by the practical difficulties hospitals face when patient, doctor, and billing records are managed separately or manually. Maintaining these records without a centralized system can lead to data duplication, inconsistency, difficulty in retrieving information, and errors in maintaining patient records. A centralized relational database provides a systematic approach to storing related hospital data and enforcing relationships through keys and constraints.

- Centralizing hospital records — patients, doctors, departments, rooms, and billing — in one relational database.
- Reducing duplicate and inconsistent information across departments.
- Maintaining referential integrity using Primary and Foreign Keys.
- Supporting faster retrieval of patient, appointment, and billing information through SQL queries.
- Providing a foundation for appointment scheduling, admission and room management, and prescription tracking.

1.3 Objectives of the Project

- Identify the major entities involved in hospital management.
- Identify suitable attributes for each entity.
- Establish relationships between the identified entities.
- Design an Entity–Relationship (ER) diagram.
- Convert the ER model into a relational database schema.
- Identify suitable Primary Keys and Foreign Keys.
- Reduce data duplication and maintain data consistency.
- Provide an organized database structure for efficient hospital record management.

1.4 Scope of the Project

The scope of the project covers the database-oriented management of core hospital information. The system connects patient details and contact information (Patient), doctor profile, specialization and department (Doctor), department name and hospital location (Department), patient–doctor appointment date and time (Appointment), patient admission, room allocation and status (Admission), room number, type and availability (Room), patient treatment, diagnosis and cost (Treatment), prescription, dosage and doctor/patient reference (Prescription), medicine name, category and price (Medicine), and patient bill, amount, date and payment status (Billing). The database can serve as the backend for a hospital administration application and can be extended with modules such as staff management, insurance processing, laboratory records, and pharmacy inventory.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual, paper-based maintenance of patient and billing records.
- Multiple Excel sheets or disconnected files for patients, doctors, and billing.
- Duplicate records and inconsistent information across departments.
- Time-consuming searches for patient history, admissions, or bills.
- Difficulty in generating consolidated reports for hospital administration.
- Higher possibility of data-entry errors and lost records.

2.2 Proposed System

The proposed system uses a centralized relational database that connects hospital records through well-defined entities, relationships, Primary Keys, and Foreign Keys. It organizes hospital information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints to enforce valid data.

- Centralized hospital database with ten major entities.
- Patient registration and record management.
- Doctor and department management.
- Appointment scheduling between patients and doctors.
- Admission and room allocation management.
- Treatment, prescription, and medicine tracking.
- Billing and payment-status management.
- SQL-based retrieval and reporting.
- Referential integrity across all related tables.

2.3 Functional Requirements

Module	Functional Requirement
Patient Management	Register, update, search, and display patient details.
Doctor Management	Store doctor profile, specialization, and associate doctors with departments.
Department Management	Store and maintain department name and location.
Appointment Management	Schedule and maintain patient–doctor appointments with date and time.
Admission & Room Management	Allocate rooms to patients on admission and track admission status.
Treatment Management	Record treatments given to patients along with the treating doctor and cost.
Prescription & Medicine Management	Maintain prescriptions with dosage, linked to a patient, doctor, and medicine.
Billing Management	Generate and track patient bills, amounts, and payment status.
Search and Reporting	Retrieve patient, doctor, admission, and billing information using SQL queries.

2.4 Non-Functional Requirements

- Performance – queries on patient, appointment, and billing data should return results efficiently.
- Integrity – invalid references (e.g. a room that does not exist) and duplicate key values must be prevented.
- Security – access to sensitive patient and billing information should be controlled for authorized users.
- Usability – the database structure should be understandable and easy for administrative staff to use.
- Reliability – transactions such as admission and billing should preserve a consistent database state.
- Scalability – the design should support growth in patients, doctors, and hospital records over time.

2.5 Requirements Specification

Category	Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	MySQL Relational Database Management System.

3. DATABASE DESIGN

3.1 Major Entities

Ten major entities have been identified for the proposed Hospital Management Database System, as summarised in Table 1.

Table 1: Major Database Entities

Entity	Description
DEPARTMENT	Stores department name and hospital location.
PATIENT	Stores patient details and contact information.
MEDICINE	Stores medicine name, category, and price.
ROOM	Stores room number, room type, and availability status.
DOCTOR	Stores doctor profile, specialization, and department.
APPOINTMENT	Represents a patient–doctor appointment with date and time.
ADMISSION	Represents a patient's admission, room allocation, and status.
TREATMENT	Stores a patient's treatment, treating doctor, and cost.
PRESCRIPTION	Stores a prescription's dosage and doctor/patient/medicine reference.
BILLING	Stores a patient's bill, amount, date, and payment status.

3.2 ER Model

The ER model represents the major hospital entities and the relationships between them. DEPARTMENT is the parent of both DOCTOR and ROOM. PATIENT and DOCTOR together drive most of the remaining relationships: a patient may have many APPOINTMENTs, ADMISSIONs, TREATMENTs, PRESCRIPTIONs, and BILLING records, while a doctor may attend many APPOINTMENTs, provide many TREATMENTs, and issue many PRESCRIPTIONs. ADMISSION links a PATIENT to a ROOM, and BILLING optionally links back to an ADMISSION for inpatient bills, while outpatient bills reference only the PATIENT. PRESCRIPTION additionally references MEDICINE.

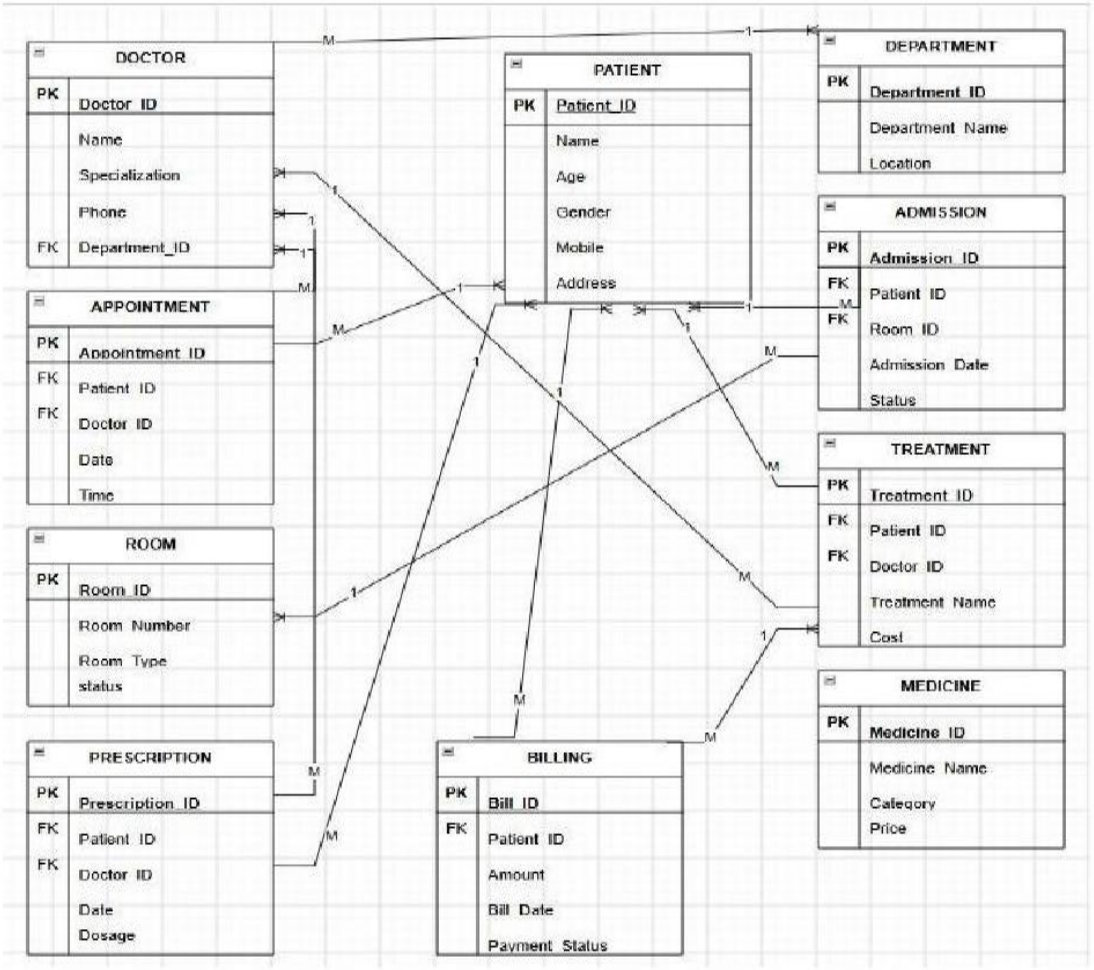
3.3 Relationships and Cardinality

Table 2: Entity Relationships and Cardinality

Relationship	Cardinality	Meaning
DEPARTMENT – DOCTOR	1 : M	One department can have many doctors.
DEPARTMENT – ROOM	1 : M	One department can contain many rooms.
DOCTOR – APPOINTMENT	1 : M	One doctor can have many appointments.
PATIENT – APPOINTMENT	1 : M	One patient can have many appointments.
PATIENT – ADMISSION	1 : M	One patient can have many admission records.
ROOM – ADMISSION	1 : M	A room can be associated with many admissions over time.
PATIENT – TREATMENT	1 : M	One patient can have many treatments.
DOCTOR – TREATMENT	1 : M	One doctor can provide many treatments.
PATIENT – PRESCRIPTION	1 : M	One patient can have many prescriptions.
DOCTOR – PRESCRIPTION	1 : M	One doctor can issue many prescriptions.
MEDICINE – PRESCRIPTION	1 : M	One medicine can appear in many prescription records.
PATIENT – BILLING	1 : M	One patient can have many bills.
ADMISSION – BILLING	1 : M	An admission can be associated with a billing record; outpatient bills leave this reference NULL.

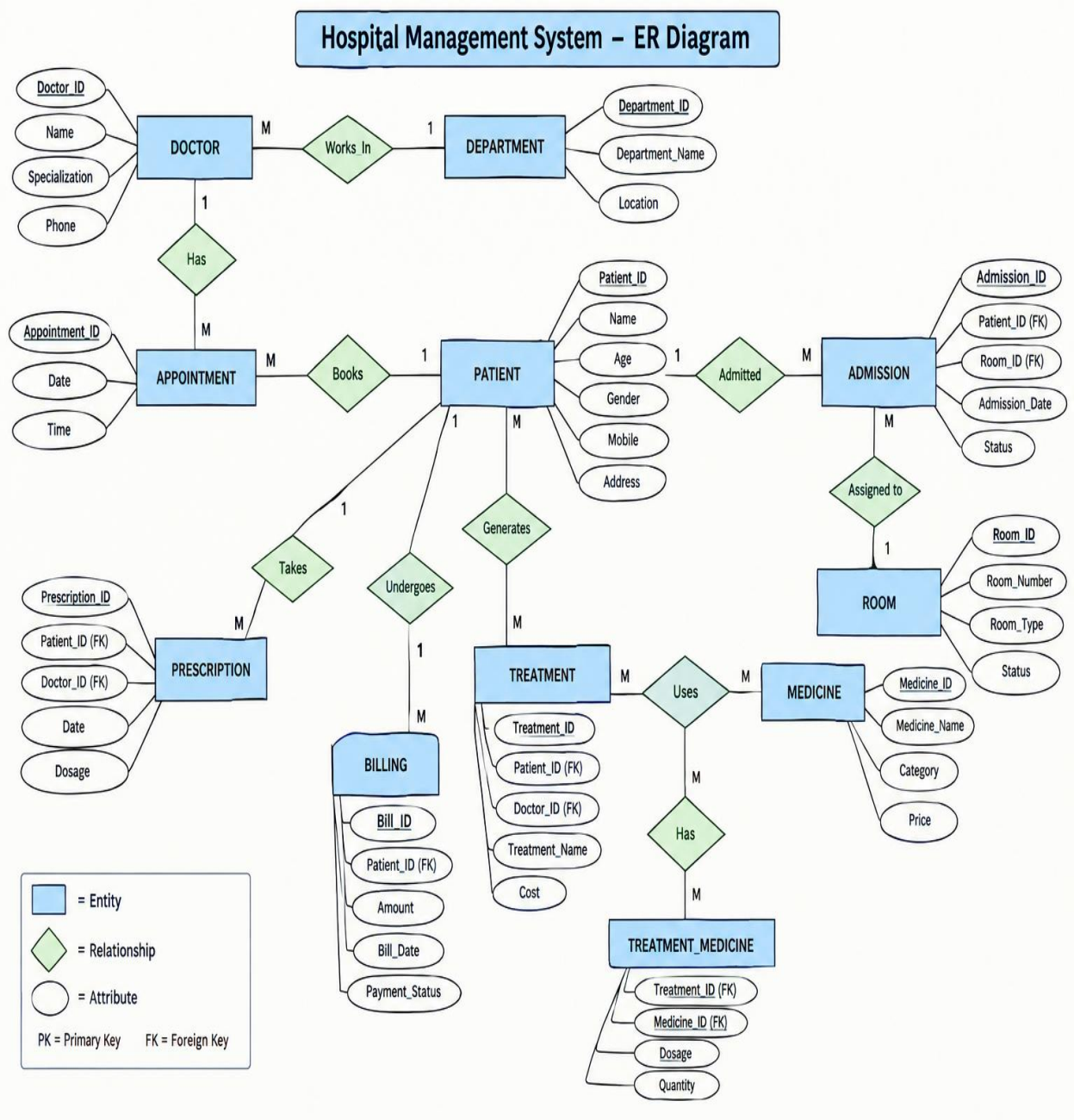
4. ER DIAGRAM

Figure 1: ER Diagram of the Hospital Management Database System(SOFTWARE BASED)



The ER diagram shows the ten entities of the Hospital Management Database System and the one-to-many relationships between them. Primary Keys (PK) and Foreign Keys (FK) are marked within each entity box, consistent with the relational schema and the SQL implementation presented in the following sections.

Figure 2: ER Diagram of the Hospital Management Database System (MANUAL)



5. RELATIONAL SCHEMA

The relational schema below follows the table-creation order used in the SQL implementation, so that every Foreign Key already has a valid parent table defined before it.

Table 3: Relational Schema Summary

Relation	Relational Schema	Keys
DEPARTMENT	Department(Department_ID, Department_Name, Location)	PK: Department_ID
PATIENT	Patient(Patient_ID, Name, Age, Gender, Mobile, Address)	PK: Patient_ID
MEDICINE	Medicine(Medicine_ID, Medicine_Name, Category, Price)	PK: Medicine_ID
ROOM	Room(Room_ID, Room_Number, Room_Type, Status, Department_ID)	PK: Room_ID; FK: Department_ID
DOCTOR	Doctor(Doctor_ID, Name, Specialization, Phone, Department_ID)	PK: Doctor_ID; FK: Department_ID
APPOINTMENT	Appointment(Appointment_ID, Patient_ID, Doctor_ID, Date, Time)	PK: Appointment_ID; FK: Patient_ID, Doctor_ID
ADMISSION	Admission(Admission_ID, Patient_ID, Room_ID, Admission_Date, Status)	PK: Admission_ID; FK: Patient_ID, Room_ID
TREATMENT	Treatment(Treatment_ID, Patient_ID, Doctor_ID, Treatment_Name, Cost)	PK: Treatment_ID; FK: Patient_ID, Doctor_ID
PRESCRIPTION	Prescription(Prescription_ID, Patient_ID, Doctor_ID, Medicine_ID, Date, Dosage)	PK: Prescription_ID; FK: Patient_ID, Doctor_ID, Medicine_ID
BILLING	Billing(Bill_ID, Patient_ID, Admission_ID, Amount, Bill_Date, Payment_Status)	PK: Bill_ID; FK: Patient_ID, Admission_ID

The Room and Doctor relations reference Department; Appointment references Patient and Doctor; Admission references Patient and Room; Treatment references Patient and Doctor; Prescription references Patient, Doctor, and Medicine; and Billing references Patient and, where applicable, Admission (the Admission_ID in Billing is nullable, since outpatient bills are not tied to a room admission).

6. TABLE DESIGN

6.1 Department Table

Table 4: Department Table Structure

Attribute	Data Type	Constraint	Description
Department_ID	INT	PK, AUTO_INCREMENT	Unique department identifier
Department_Name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the department
Location	VARCHAR(100)	NOT NULL	Physical location of the department

6.2 Patient Table

Table 5: Patient Table Structure

Attribute	Data Type	Constraint	Description
Patient_ID	INT	PK, AUTO_INCREMENT	Unique patient identifier
Name	VARCHAR(100)	NOT NULL	Patient's full name
Age	INT	NOT NULL, CHECK (0–120)	Patient's age
Gender	ENUM('Male', 'Female', 'Other')	NOT NULL	Patient's gender
Mobile	VARCHAR(15)	NOT NULL, UNIQUE	Patient's contact number
Address	VARCHAR(255)	NOT NULL	Patient's residential address

6.3 Medicine Table

Table 6: Medicine Table Structure

Attribute	Data Type	Constraint	Description
Medicine_ID	INT	PK, AUTO_INCREMENT	Unique medicine identifier
Medicine_Name	VARCHAR(100)	NOT NULL	Name of the medicine
Category	VARCHAR(50)	NOT NULL	Medicine category (e.g. Analgesic, Antibiotic)
Price	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)	Unit price of the medicine

6.4 Room Table

Table 7: Room Table Structure

Attribute	Data Type	Constraint	Description
Room_ID	INT	PK, AUTO_INCREMENT	Unique room identifier
Room_Number	VARCHAR(10)	NOT NULL, UNIQUE	Room number
Room_Type	ENUM('General', 'Private', 'ICU', 'Deluxe')	NOT NULL	Type/category of room
Status	ENUM('Available', 'Occupied', 'Maintenance')	NOT NULL, DEFAULT 'Available'	Current availability status
Department_ID	INT	NOT NULL, FK	References Department

6.5 Doctor Table

Table 8: Doctor Table Structure

Attribute	Data Type	Constraint	Description
Doctor_ID	INT	PK, AUTO_INCREMENT	Unique doctor identifier
Name	VARCHAR(100)	NOT NULL	Doctor's full name
Specialization	VARCHAR(100)	NOT NULL	Doctor's medical specialization
Phone	VARCHAR(15)	NOT NULL, UNIQUE	Doctor's contact number
Department_ID	INT	NOT NULL, FK	References Department

6.6 Appointment Table

Table 9: Appointment Table Structure

Attribute	Data Type	Constraint	Description
Appointment_ID	INT	PK, AUTO_INCREMENT	Unique appointment identifier
Patient_ID	INT	NOT NULL, FK	References Patient
Doctor_ID	INT	NOT NULL, FK	References Doctor
Date	DATE	NOT NULL	Appointment date
Time	TIME	NOT NULL	Appointment time

6.7 Admission Table

Table 10: Admission Table Structure

Attribute	Data Type	Constraint	Description
Admission_ID	INT	PK, AUTO_INCREMENT	Unique admission identifier
Patient_ID	INT	NOT NULL, FK	References Patient
Room_ID	INT	NOT NULL, FK	References Room
Admission_Date	DATE	NOT NULL	Date of admission
Status	ENUM('Admitted', 'Discharged')	NOT NULL, DEFAULT 'Admitted'	Current admission status

6.8 Treatment Table

Table 11: Treatment Table Structure

Attribute	Data Type	Constraint	Description
Treatment_ID	INT	PK, AUTO_INCREMENT	Unique treatment identifier
Patient_ID	INT	NOT NULL, FK	References Patient
Doctor_ID	INT	NOT NULL, FK	References Doctor
Treatment_Name	VARCHAR(150)	NOT NULL	Name/description of the treatment
Cost	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)	Cost of the treatment

6.9 Prescription Table

Table 12: Prescription Table Structure

Attribute	Data Type	Constraint	Description
Prescription_ID	INT	PK, AUTO_INCREMENT	Unique prescription identifier
Patient_ID	INT	NOT NULL, FK	References Patient
Doctor_ID	INT	NOT NULL, FK	References Doctor
Medicine_ID	INT	NOT NULL, FK	References Medicine
Date	DATE	NOT NULL	Date the prescription was issued
Dosage	VARCHAR(100)	NOT NULL	Dosage instructions

6.10 Billing Table

Table 13: Billing Table Structure

Attribute	Data Type	Constraint	Description
Bill_ID	INT	PK, AUTO_INCREMENT	Unique bill identifier
Patient_ID	INT	NOT NULL, FK	References Patient
Admission_ID	INT	NULL, FK	References Admission (NULL for outpatient bills)
Amount	DECIMAL(10,2)	NOT NULL, CHECK (>= 0)	Bill amount
Bill_Date	DATE	NOT NULL	Date the bill was raised
Payment_Status	ENUM('Paid', 'Pending', 'Partial')	NOT NULL, DEFAULT 'Pending'	Current payment status

6.11 Keys and Constraints

Table 14: Keys and Constraints Summary

Constraint Type	Applied To / Purpose
Primary Key	Department_ID, Patient_ID, Medicine_ID, Room_ID, Doctor_ID, Appointment_ID, Admission_ID, Treatment_ID, Prescription_ID, Bill_ID
Foreign Key	Room.Department_ID; Doctor.Department_ID; Appointment.Patient_ID, Appointment.Doctor_ID; Admission.Patient_ID, Admission.Room_ID; Treatment.Patient_ID, Treatment.Doctor_ID; Prescription.Patient_ID, Prescription.Doctor_ID, Prescription.Medicine_ID; Billing.Patient_ID, Billing.Admission_ID
NOT NULL	Required attributes such as names, identifiers, dates, and status values
UNIQUE	Department_Name, Patient.Mobile, Room_Number, Doctor.Phone
CHECK	Patient.Age (0–120), Medicine.Price (>= 0), Treatment.Cost (>= 0), Billing.Amount (>= 0)
DEFAULT	Room.Status ('Available'), Admission.Status ('Admitted'), Billing.Payment_Status ('Pending')
ON UPDATE / ON DELETE	CASCADE is used on Patient-owned records (Appointment, Admission, Treatment, Prescription, Billing) so a patient's dependent records follow it; RESTRICT is used where a referenced Department, Doctor, Room, or Medicine must not be removed while still in use.

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The following example uses a single, unnormalized hospital record — covering a patient's admission, treatments, and prescriptions — to illustrate the process from the Unnormalized Form (UNF) to the Third Normal Form (3NF).

7.1 Unnormalized Form (UNF)

A single hospital record might contain Patient_ID, Patient_Name, Address, Doctor_Name, Department_Name, Room_Number, together with multiple treatment values (Treatment_Name, Cost) and multiple prescription values (Medicine_Name, Dosage) for the same patient, all stored in one record. Repeating treatment and prescription groups, together with repeated doctor and department details, create redundancy.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and every attribute is made atomic. Each treatment given to a patient is represented as a separate row, and each prescription is represented as a separate row. Patient, doctor, and department details may still be repeated across these rows.

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on a composite key are removed. Patient-specific attributes (Name, Age, Gender, Mobile, Address) depend only on Patient_ID; doctor-specific attributes (Name, Specialization, Phone) depend only on Doctor_ID; and treatment- or prescription-specific attributes depend on their own Treatment_ID or Prescription_ID rather than on the combination of Patient_ID and Doctor_ID together. These dependencies motivate separate Patient, Doctor, Treatment, and Prescription relations.

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Department details (Department_Name, Location) are stored once in Department rather than being repeated in Doctor and Room. Room details are stored in Room rather than repeated in Admission. Medicine details are stored in Medicine rather than repeated in Prescription. The resulting ten relations — Department, Patient, Medicine, Room, Doctor, Appointment, Admission, Treatment, Prescription, and Billing — correspond to the tables implemented in the SQL script.

Relation	Reason for Separation
Department	Stores department-specific attributes (name, location) once, instead of repeating them for every doctor or room.
Patient	Stores patient-specific attributes independently of any appointment, admission, treatment, prescription, or bill.
Doctor	Stores doctor-specific attributes and a Department_ID reference, instead of repeating department details.
Room	Stores room-specific attributes and a Department_ID reference, instead of repeating them in every Admission record.
Medicine	Stores medicine-specific attributes once, instead of repeating them in every Prescription record.
Appointment	Stores the patient–doctor appointment relationship with its own date and time.
Admission	Stores the patient–room admission relationship with its own date and status.
Treatment	Stores treatment-specific details linked to a patient and the treating doctor.
Prescription	Stores prescription-specific details linked to a patient, doctor, and medicine.
Billing	Stores bill-specific details linked to a patient and, where applicable, an admission.

8. SQL IMPLEMENTATION

The SQL implementation below is exactly as executed against the hospital_management_db database. Tables are created in an order that satisfies every Foreign Key dependency: DEPARTMENT, PATIENT, and MEDICINE first (no dependencies), followed by ROOM and DOCTOR (depend on DEPARTMENT), and finally APPOINTMENT, ADMISSION, TREATMENT, PRESCRIPTION, and BILLING, which depend on the tables created before them.

8.1 DDL – Data Definition Language

```
CREATE DATABASE hospital_management_db;
USE hospital_management_db;
```

```
-- 1. DEPARTMENT
CREATE TABLE DEPARTMENT (
    Department_ID   INT AUTO_INCREMENT PRIMARY KEY,
    Department_Name VARCHAR(100) NOT NULL UNIQUE,
    Location        VARCHAR(100) NOT NULL
);
```

```
-- 2. PATIENT
CREATE TABLE PATIENT (
    Patient_ID  INT AUTO_INCREMENT PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    Age         INT NOT NULL CHECK (Age BETWEEN 0 AND 120),
    Gender      ENUM('Male', 'Female', 'Other') NOT NULL,
    Mobile      VARCHAR(15) NOT NULL UNIQUE,
    Address     VARCHAR(255) NOT NULL
);
```

```
-- 3. MEDICINE
CREATE TABLE MEDICINE (
    Medicine_ID   INT AUTO_INCREMENT PRIMARY KEY,
    Medicine_Name VARCHAR(100) NOT NULL,
    Category      VARCHAR(50) NOT NULL,
    Price         DECIMAL(10,2) NOT NULL CHECK (Price >= 0)
);
```

```
-- 4. ROOM (references DEPARTMENT)
CREATE TABLE ROOM (
    Room_ID       INT AUTO_INCREMENT PRIMARY KEY,
    Room_Number   VARCHAR(10) NOT NULL UNIQUE,
    Room_Type     ENUM('General', 'Private', 'ICU', 'Deluxe') NOT NULL,
    Status        ENUM('Available', 'Occupied', 'Maintenance') NOT NULL DEFAULT 'Available',
    Department_ID INT NOT NULL,
    CONSTRAINT fk_room_department
        FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 5. DOCTOR (references DEPARTMENT)
CREATE TABLE DOCTOR (
    Doctor_ID      INT AUTO_INCREMENT PRIMARY KEY,
    Name           VARCHAR(100) NOT NULL,
    Specialization VARCHAR(100) NOT NULL,
    Phone          VARCHAR(15) NOT NULL UNIQUE,
    Department_ID  INT NOT NULL,
    CONSTRAINT fk_doctor_department
```

```
        FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 6. APPOINTMENT (references PATIENT, DOCTOR)
CREATE TABLE APPOINTMENT (
    Appointment_ID INT AUTO_INCREMENT PRIMARY KEY,
    Patient_ID     INT NOT NULL,
    Doctor_ID      INT NOT NULL,
    Date           DATE NOT NULL,
    Time           TIME NOT NULL,
    CONSTRAINT fk_appointment_patient
        FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_appointment_doctor
        FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(Doctor_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 7. ADMISSION (references PATIENT, ROOM)
CREATE TABLE ADMISSION (
    Admission_ID   INT AUTO_INCREMENT PRIMARY KEY,
    Patient_ID     INT NOT NULL,
    Room_ID        INT NOT NULL,
    Admission_Date DATE NOT NULL,
    Status         ENUM('Admitted', 'Discharged') NOT NULL DEFAULT 'Admitted',
    CONSTRAINT fk_admission_patient
        FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_admission_room
        FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 8. TREATMENT (references PATIENT, DOCTOR)
CREATE TABLE TREATMENT (
    Treatment_ID   INT AUTO_INCREMENT PRIMARY KEY,
    Patient_ID     INT NOT NULL,
    Doctor_ID      INT NOT NULL,
    Treatment_Name VARCHAR(150) NOT NULL,
    Cost           DECIMAL(10,2) NOT NULL CHECK (Cost >= 0),
    CONSTRAINT fk_treatment_patient
        FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_treatment_doctor
        FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(Doctor_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 9. PRESCRIPTION (references PATIENT, DOCTOR, MEDICINE)
CREATE TABLE PRESCRIPTION (
    Prescription_ID INT AUTO_INCREMENT PRIMARY KEY,
    Patient_ID      INT NOT NULL,
    Doctor_ID       INT NOT NULL,
    Medicine_ID     INT NOT NULL,
    Date            DATE NOT NULL,
    Dosage          VARCHAR(100) NOT NULL,
    CONSTRAINT fk_prescription_patient
        FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_prescription_doctor
        FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(Doctor_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_prescription_medicine
        FOREIGN KEY (Medicine_ID) REFERENCES MEDICINE(Medicine_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
-- 10. BILLING (references PATIENT, ADMISSION)
CREATE TABLE BILLING (
    Bill_ID        INT AUTO_INCREMENT PRIMARY KEY,
    Patient_ID     INT NOT NULL,
    Admission_ID   INT NULL,
    Amount         DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
    Bill_Date      DATE NOT NULL,
    Payment_Status ENUM('Paid', 'Pending', 'Partial') NOT NULL DEFAULT 'Pending',
    CONSTRAINT fk_billing_patient
        FOREIGN KEY (Patient_ID) REFERENCES PATIENT(Patient_ID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_billing_admission
        FOREIGN KEY (Admission_ID) REFERENCES ADMISSION(Admission_ID)
        ON UPDATE CASCADE ON DELETE CASCADE
);
-- Admission_ID is nullable because not every billed encounter involves
-- a room admission (e.g., outpatient consultations/checkups).
```

8.2 DML – Data Manipulation Language

Each table is populated with a minimum of five FK-consistent sample rows. Representative INSERT statements are shown below, along with the CRUD demonstration statements used for the review panel.

```
-- DEPARTMENT (5 rows)
INSERT INTO DEPARTMENT (Department_Name, Location) VALUES
('Cardiology',       'Block A, Floor 1'),
('Orthopedics',      'Block A, Floor 2'),
('Neurology',        'Block B, Floor 1'),
('Pediatrics',       'Block B, Floor 2'),
('General Medicine', 'Block C, Floor 1');
```

```
-- PATIENT (6 rows)
INSERT INTO PATIENT (Name, Age, Gender, Mobile, Address) VALUES
('Ravi Kumar',     34, 'Male',   '9876543210', 'Kakinada, AP'),
('Sita Devi',      28, 'Female', '9876543211', 'Rajahmundry, AP'),
('Anil Reddy',     45, 'Male',   '9876543212', 'Vijayawada, AP'),
('Lakshmi Priya',  52, 'Female', '9876543213', 'Visakhapatnam, AP'),
('Mohammed Imran', 19, 'Male',   '9876543214', 'Eluru, AP'),
('Kavitha Rao',    63, 'Female', '9876543215', 'Guntur, AP');
```

```
-- MEDICINE, ROOM, DOCTOR, APPOINTMENT, ADMISSION, TREATMENT,
-- PRESCRIPTION and BILLING are populated the same way, with five or
-- more FK-consistent rows each. Representative rows:
INSERT INTO MEDICINE (Medicine_Name, Category, Price) VALUES
('Paracetamol 500mg', 'Analgesic', 15.50);


INSERT INTO ROOM (Room_Number, Room_Type, Status, Department_ID) VALUES
('101', 'General', 'Occupied', 1);


INSERT INTO DOCTOR (Name, Specialization, Phone, Department_ID) VALUES
('Dr. Suresh Babu', 'Cardiologist', '9000011111', 1);
```

```
INSERT INTO APPOINTMENT (Patient_ID, Doctor_ID, Date, Time) VALUES
(1, 1, '2026-09-01', '09:30:00');

INSERT INTO ADMISSION (Patient_ID, Room_ID, Admission_Date, Status) VALUES
(1, 1, '2026-09-01', 'Admitted');

INSERT INTO TREATMENT (Patient_ID, Doctor_ID, Treatment_Name, Cost) VALUES
(1, 1, 'Angioplasty', 125000.00);

INSERT INTO PRESCRIPTION (Patient_ID, Doctor_ID, Medicine_ID, Date, Dosage) VALUES
(1, 1, 3, '2026-09-01', 'Atorvastatin 10mg - 1 tab OD after dinner');

INSERT INTO BILLING (Patient_ID, Admission_ID, Amount, Bill_Date, Payment_Status) VALUES
(1, 1, 125000.00, '2026-09-05', 'Partial');
```

```
-- CRUD demonstration for the review panel
-- (a) INSERT: Register a new patient
INSERT INTO PATIENT (Name, Age, Gender, Mobile, Address)
VALUES ('Deepak Sinha', 40, 'Male', '9876543299', 'Kakinada, AP');

-- (c) UPDATE: Mark a room as available after patient discharge
UPDATE ROOM SET Status = 'Available' WHERE Room_ID = 2;

-- (d) DELETE: Remove a cancelled appointment
DELETE FROM APPOINTMENT WHERE Appointment_ID = 5;
```

8.3 DQL – Data Query Language

A few basic retrieval queries are shown here; the full query set with output placeholders is presented in Section 9.

```
-- (b) SELECT with WHERE: Find all patients aged above 40
SELECT Patient_ID, Name, Age, Gender
FROM PATIENT
WHERE Age > 40;
```

```
-- EQUI JOIN: Doctor names along with their Department names
SELECT D.Name AS Doctor_Name, D.Specialization, Dept.Department_Name
FROM DOCTOR D
JOIN DEPARTMENT Dept
    ON D.Department_ID = Dept.Department_ID;
```

```
-- LEFT JOIN: All patients with admission details, if any
SELECT P.Patient_ID, P.Name, A.Admission_Date, A.Status
FROM PATIENT P
LEFT JOIN ADMISSION A
    ON P.Patient_ID = A.Patient_ID;
```

9. SQL QUERIES & SCREENSHOTS

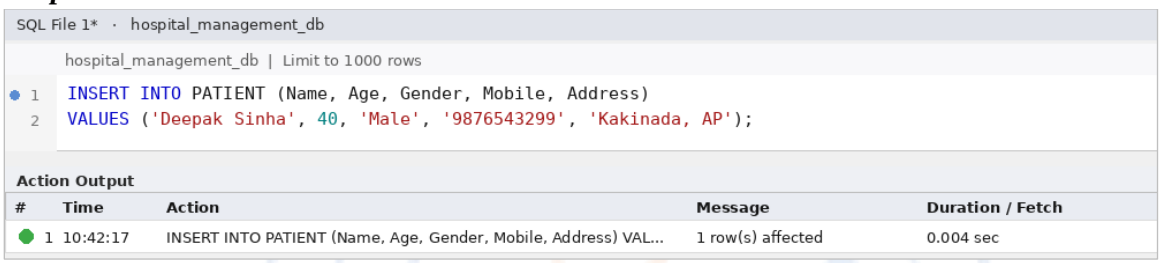
The queries below are grouped by category — CRUD operations, joins, aggregation, a subquery, and a view — followed by additional retrieval queries built on the same schema. A screenshot is provided after each query for the actual MySQL output to be inserted before final submission.

9.1 CRUD Operations

Q1. Register a new patient (INSERT)

```
INSERT INTO PATIENT (Name, Age, Gender, Mobile, Address)
VALUES ('Deepak Sinha', 40, 'Male', '9876543299', 'Kakinada, AP');
```

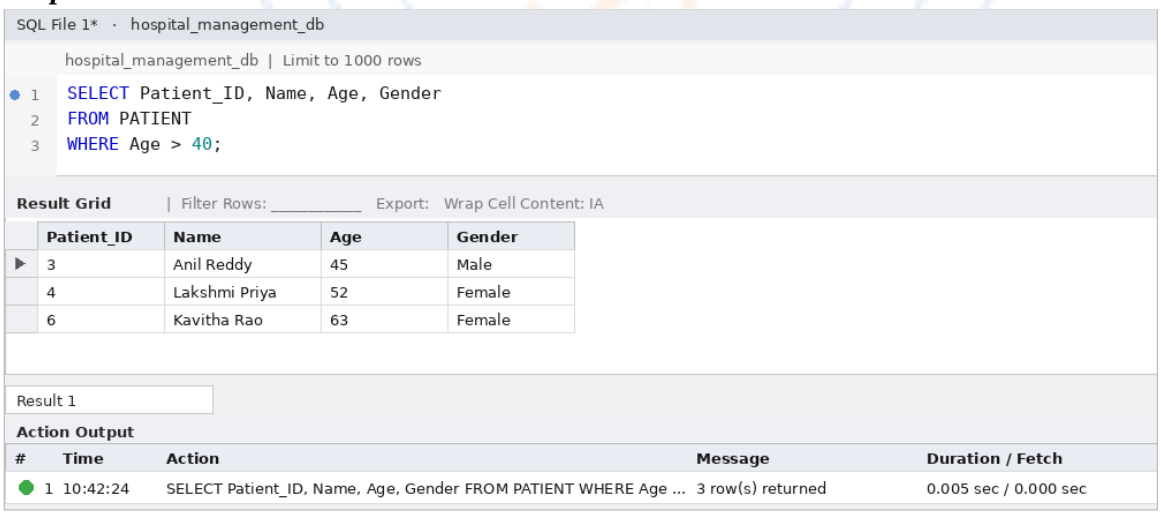
Output:



Q2. Find all patients aged above 40 (SELECT ... WHERE)

```
SELECT Patient_ID, Name, Age, Gender
FROM PATIENT
WHERE Age > 40;
```

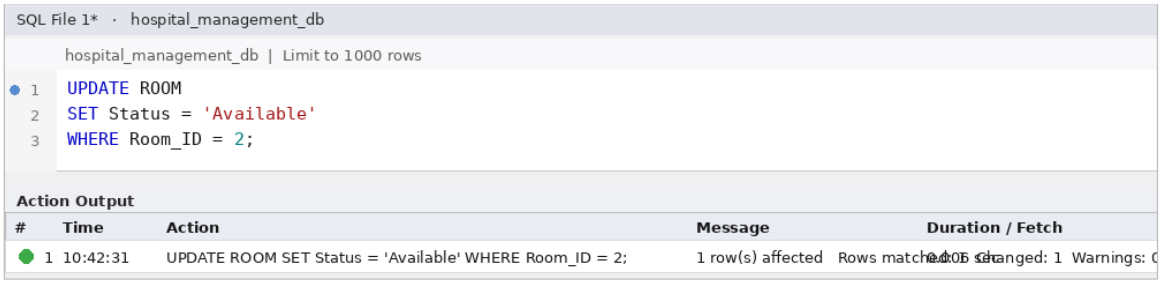
Output:



Q3. Mark a room as available after discharge (UPDATE)

```
UPDATE ROOM
SET Status = 'Available'
WHERE Room_ID = 2;
```

Output:



Q4. Remove a cancelled appointment (DELETE)

```
DELETE FROM APPOINTMENT
WHERE Appointment_ID = 5;
```

Output:

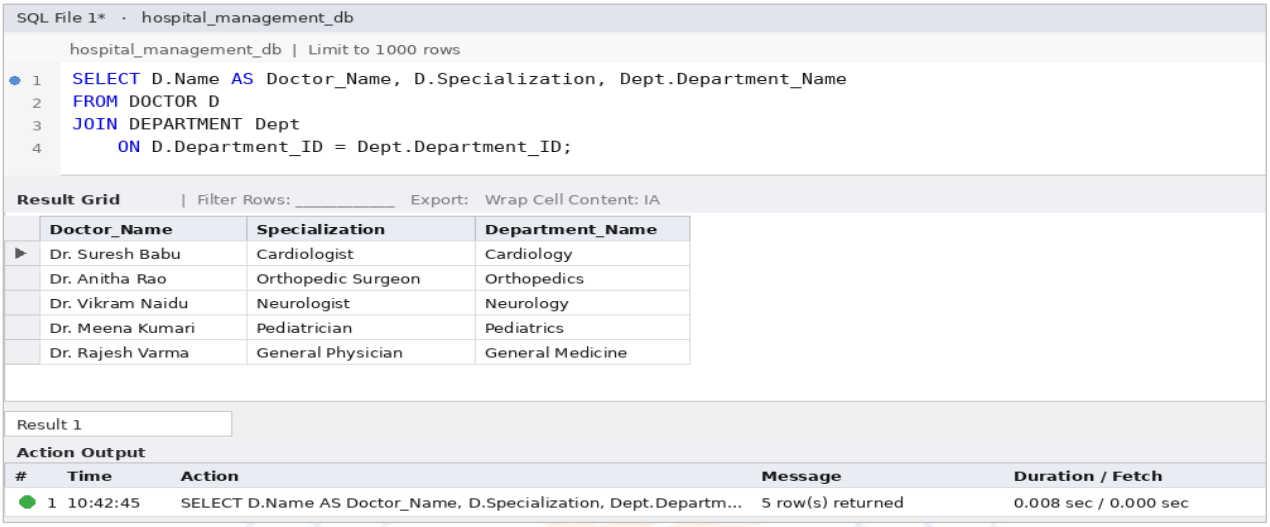


9.2 Joins

Q5. Doctor names along with their department names (EQUI JOIN)

```
SELECT D.Name AS Doctor_Name, D.Specialization, Dept.Department_Name
FROM DOCTOR D
JOIN DEPARTMENT Dept
    ON D.Department_ID = Dept.Department_ID;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT D.Name AS Doctor_Name, D.Specialization, Dept.Department_Name
2  FROM DOCTOR D
3  JOIN DEPARTMENT Dept
4      ON D.Department_ID = Dept.Department_ID;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Doctor_Name	Specialization	Department_Name
Dr. Suresh Babu	Cardiologist	Cardiology
Dr. Anitha Rao	Orthopedic Surgeon	Orthopedics
Dr. Vikram Naidu	Neurologist	Neurology
Dr. Meena Kumari	Pediatrician	Pediatrics
Dr. Rajesh Varma	General Physician	General Medicine

Result 1

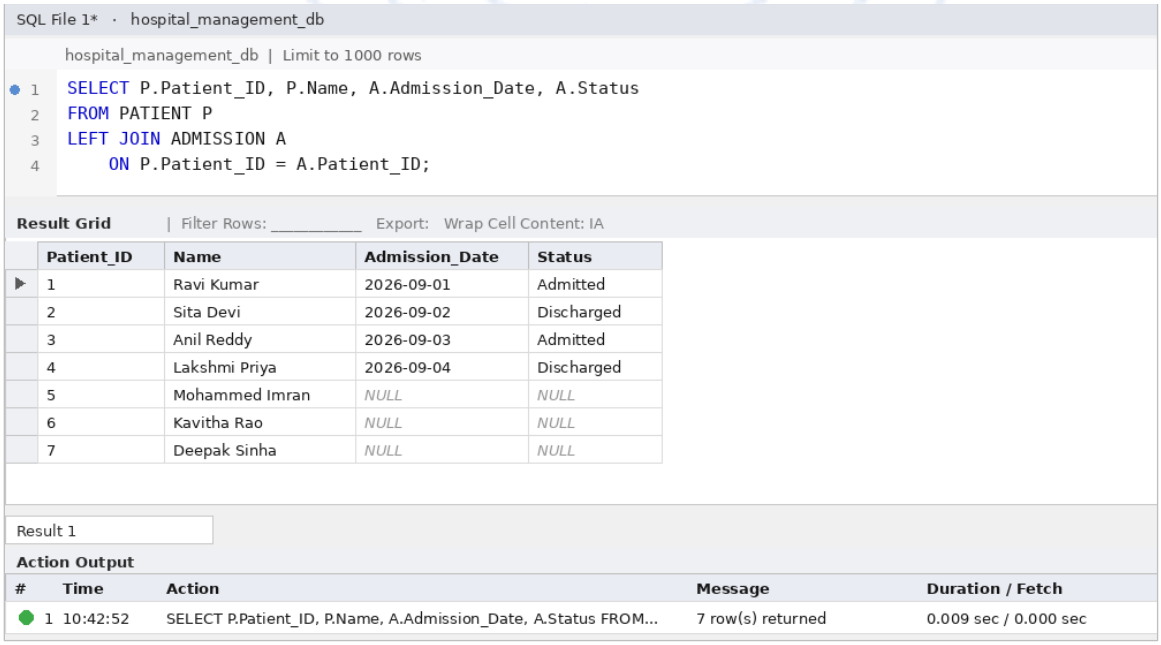
Action Output

#	Time	Action	Message	Duration / Fetch
1	10:42:45	SELECT D.Name AS Doctor_Name, D.Specialization, Dept.Departm...	5 row(s) returned	0.008 sec / 0.000 sec

Q6. All patients with admission details, including those never admitted (LEFT JOIN)

```
SELECT P.Patient_ID, P.Name, A.Admission_Date, A.Status
FROM PATIENT P
LEFT JOIN ADMISSION A
    ON P.Patient_ID = A.Patient_ID;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT P.Patient_ID, P.Name, A.Admission_Date, A.Status
2  FROM PATIENT P
3  LEFT JOIN ADMISSION A
4      ON P.Patient_ID = A.Patient_ID;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Patient_ID	Name	Admission_Date	Status
1	Ravi Kumar	2026-09-01	Admitted
2	Sita Devi	2026-09-02	Discharged
3	Anil Reddy	2026-09-03	Admitted
4	Lakshmi Priya	2026-09-04	Discharged
5	Mohammed Imran	NULL	NULL
6	Kavitha Rao	NULL	NULL
7	Deepak Sinha	NULL	NULL

Result 1

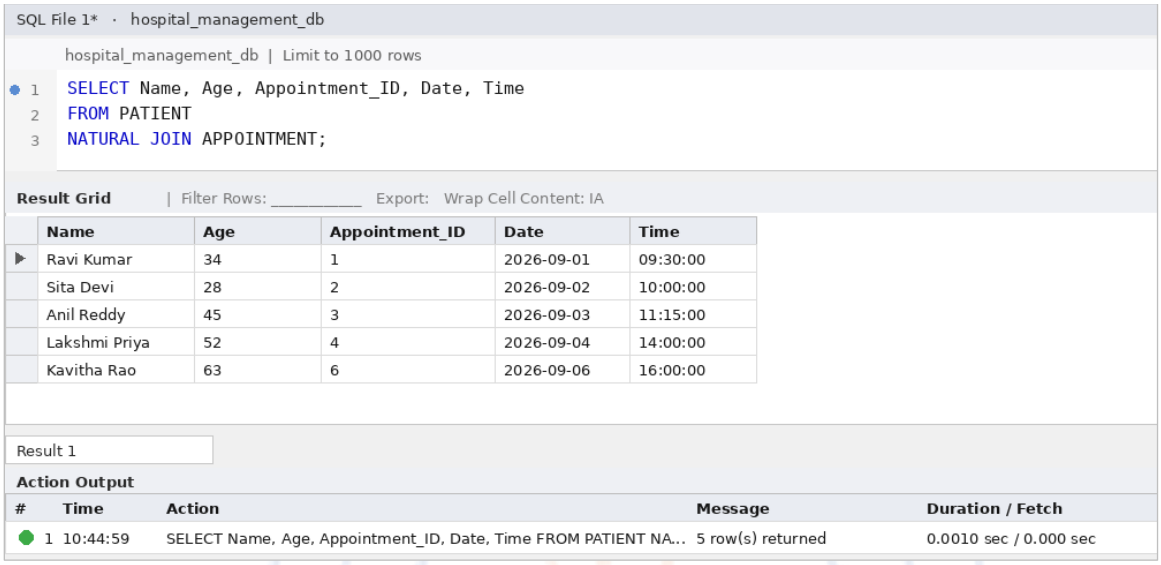
Action Output

#	Time	Action	Message	Duration / Fetch
1	10:42:52	SELECT P.Patient_ID, P.Name, A.Admission_Date, A.Status FROM...	7 row(s) returned	0.009 sec / 0.000 sec

Q7. Appointments matched with patient details (NATURAL JOIN)

```
SELECT Name, Age, Appointment_ID, Date, Time
FROM PATIENT
NATURAL JOIN APPOINTMENT;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Name, Age, Appointment_ID, Date, Time
2  FROM PATIENT
3  NATURAL JOIN APPOINTMENT;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Name	Age	Appointment_ID	Date	Time
Ravi Kumar	34	1	2026-09-01	09:30:00
Sita Devi	28	2	2026-09-02	10:00:00
Anil Reddy	45	3	2026-09-03	11:15:00
Lakshmi Priya	52	4	2026-09-04	14:00:00
Kavitha Rao	63	6	2026-09-06	16:00:00

Result 1

Action Output

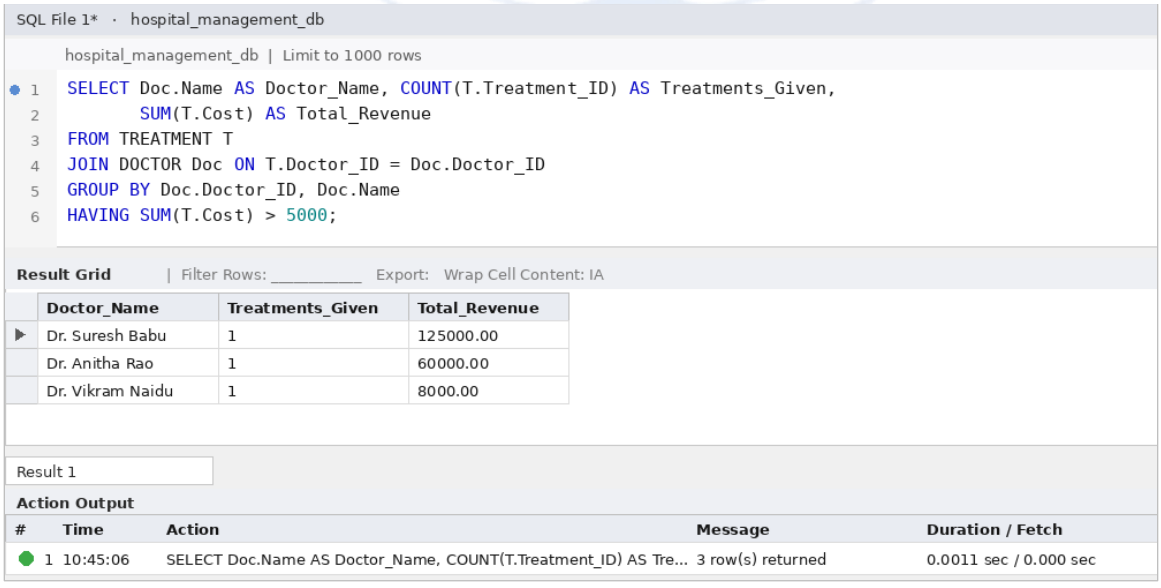
#	Time	Action	Message	Duration / Fetch
1	10:44:59	SELECT Name, Age, Appointment_ID, Date, Time FROM PATIENT NA...	5 row(s) returned	0.0010 sec / 0.000 sec

9.3 Aggregation & Grouping

Q8. Doctors whose total treatment revenue exceeds 5000

```
SELECT Doc.Name AS Doctor_Name, COUNT(T.Treatment_ID) AS Treatments_Given,
       SUM(T.Cost) AS Total_Revenue
FROM TREATMENT T
JOIN DOCTOR Doc ON T.Doctor_ID = Doc.Doctor_ID
GROUP BY Doc.Doctor_ID, Doc.Name
HAVING SUM(T.Cost) > 5000;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Doc.Name AS Doctor_Name, COUNT(T.Treatment_ID) AS Treatments_Given,
2         SUM(T.Cost) AS Total_Revenue
3  FROM TREATMENT T
4  JOIN DOCTOR Doc ON T.Doctor_ID = Doc.Doctor_ID
5  GROUP BY Doc.Doctor_ID, Doc.Name
6  HAVING SUM(T.Cost) > 5000;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Doctor_Name	Treatments_Given	Total_Revenue
Dr. Suresh Babu	1	125000.00
Dr. Anitha Rao	1	60000.00
Dr. Vikram Naidu	1	8000.00

Result 1

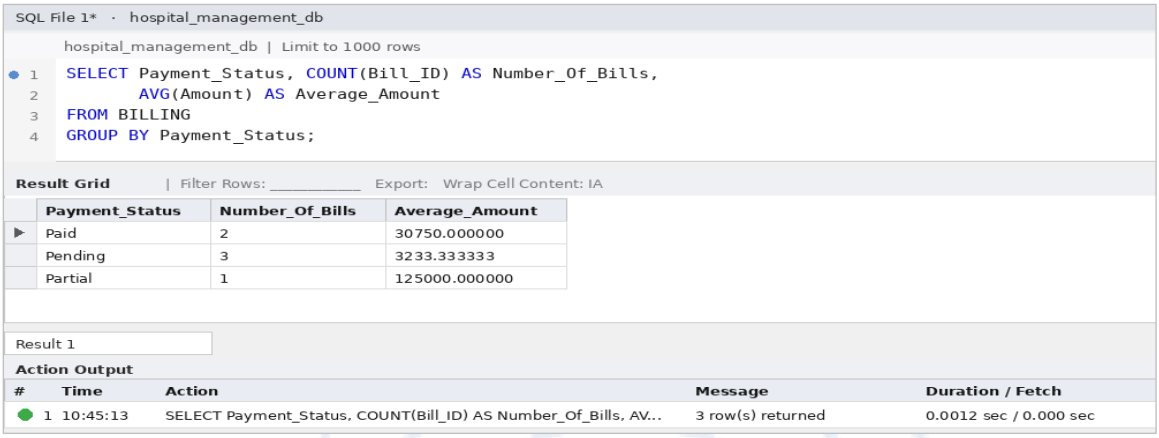
Action Output

#	Time	Action	Message	Duration / Fetch
1	10:45:06	SELECT Doc.Name AS Doctor_Name, COUNT(T.Treatment_ID) AS Tre...	3 row(s) returned	0.0011 sec / 0.000 sec

Q9. Average billing amount per payment status

```
SELECT Payment_Status, COUNT(Bill_ID) AS Number_Of_Bills,
       AVG(Amount) AS Average_Amount
FROM BILLING
GROUP BY Payment_Status;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Payment_Status, COUNT(Bill_ID) AS Number_Of_Bills,
2         AVG(Amount) AS Average_Amount
3  FROM BILLING
4  GROUP BY Payment_Status;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Payment_Status	Number_Of_Bills	Average_Amount
Paid	2	30750.000000
Pending	3	3233.333333
Partial	1	125000.000000

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:45:13	SELECT Payment_Status, COUNT(Bill_ID) AS Number_Of_Bills, AV...	3 row(s) returned	0.0012 sec / 0.000 sec

9.4 Subquery

Q10. Patients whose total billing amount exceeds the average (correlated subquery)

```
SELECT P.Patient_ID, P.Name
FROM PATIENT P
WHERE (
    SELECT SUM(B.Amount)
    FROM BILLING B
    WHERE B.Patient_ID = P.Patient_ID
) > (
    SELECT AVG(Amount) FROM BILLING
);
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT P.Patient_ID, P.Name
2  FROM PATIENT P
3  WHERE (
4      SELECT SUM(B.Amount)
5      FROM BILLING B
6      WHERE B.Patient_ID = P.Patient_ID
7  ) > (
8      SELECT AVG(Amount) FROM BILLING
9  );
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Patient_ID	Name
1	Ravi Kumar
2	Sita Devi

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:45:20	SELECT P.Patient_ID, P.Name FROM PATIENT P WHERE (SELECT SU...	2 row(s) returned	0.0013 sec / 0.000 sec

9.5 View

Q11. Create a view joining Patient, Admission, and Room

```
CREATE VIEW Patient_Admission_Room_View AS
SELECT
    P.Patient_ID,
    P.Name           AS Patient_Name,
    P.Age,
    P.Gender,
    A.Admission_ID,
    A.Admission_Date,
    A.Status         AS Admission_Status,
    R.Room_ID,
    R.Room_Number,
    R.Room_Type,
    R.Status         AS Room_Status
FROM PATIENT P
JOIN ADMISSION A ON P.Patient_ID = A.Patient_ID
JOIN ROOM R       ON A.Room_ID = R.Room_ID;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  CREATE VIEW Patient_Admission_Room_View AS
2  SELECT
3      P.Patient_ID,
4      P.Name            AS Patient_Name,
5      P.Age,
6      P.Gender,
7      A.Admission_ID,
8      A.Admission_Date,
9      A.Status          AS Admission_Status,
10     R.Room_ID,
11     R.Room_Number,
12     R.Room_Type,
13     R.Status          AS Room_Status
14 FROM PATIENT P
15 JOIN ADMISSION A ON P.Patient_ID = A.Patient_ID
16 JOIN ROOM R        ON A.Room_ID = R.Room_ID;
```

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:45:27	CREATE VIEW Patient_Admission_Room_View AS SELECT P.Patient_...	0 row(s) affected	0.0014 sec

Q12. Query the view

```
SELECT * FROM Patient_Admission_Room_View;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT * FROM Patient_Admission_Room_View;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Patient_ID	Patient_Name	Age	Gender	Admission_ID	Admission_Date	Admission_Status	Room_ID	Room_Number	Room_Type	Room_Status
1	Ravi Kumar	34	Male	1	2026-09-01	Admitted	1	101	General	Occupied
2	Sita Devi	28	Female	2	2026-09-02	Discharged	2	102	Private	Available
3	Anil Reddy	45	Male	3	2026-09-03	Admitted	3	201	ICU	Occupied
4	Lakshmi Priya	52	Female	4	2026-09-04	Discharged	4	202	Deluxe	Available

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:46:34	SELECT * FROM Patient_Admission_Room_View;	4 row(s) returned	0.0015 sec / 0.000 sec

9.6 Additional Queries

Q13. List all departments

```
SELECT * FROM DEPARTMENT;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT * FROM DEPARTMENT;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Department_ID	Department_Name	Location
1	Cardiology	Block A, Floor 1
2	Orthopedics	Block A, Floor 2
3	Neurology	Block B, Floor 1
4	Pediatrics	Block B, Floor 2
5	General Medicine	Block C, Floor 1

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:46:41	SELECT * FROM DEPARTMENT;	5 row(s) returned	0.0016 sec / 0.000 sec

Q14. List all doctors with their specialization

```
SELECT Doctor_ID, Name, Specialization
FROM DOCTOR;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Doctor_ID, Name, Specialization
2  FROM DOCTOR;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Doctor_ID	Name	Specialization
1	Dr. Suresh Babu	Cardiologist
2	Dr. Anitha Rao	Orthopedic Surgeon
3	Dr. Vikram Naidu	Neurologist
4	Dr. Meena Kumari	Pediatrician
5	Dr. Rajesh Varma	General Physician

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:46:48	SELECT Doctor_ID, Name, Specialization FROM DOCTOR;	5 row(s) returned	0.0017 sec / 0.000 sec

Q15. List rooms that are currently available

```
SELECT * FROM ROOM
WHERE Status = 'Available';
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT * FROM ROOM
2  WHERE Status = 'Available';
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Room_ID	Room_Number	Room_Type	Status	Department_ID
2	102	Private	Available	2
4	202	Deluxe	Available	4

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:46:55	SELECT * FROM ROOM WHERE Status = 'Available';	2 row(s) returned	0.0018 sec / 0.000 sec

Q16. List patients who are currently admitted

```
SELECT P.Name, A.Admission_Date
FROM PATIENT P
JOIN ADMISSION A ON P.Patient_ID = A.Patient_ID
WHERE A.Status = 'Admitted';
```

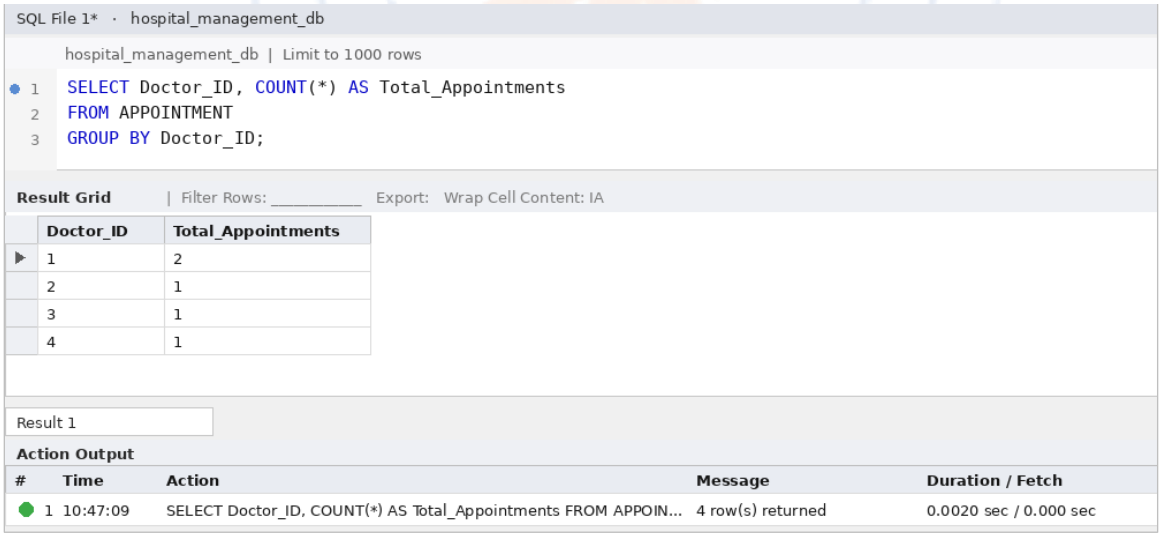
Output:



Q17. Total number of appointments handled by each doctor

```
SELECT Doctor_ID, COUNT(*) AS Total_Appointments
FROM APPOINTMENT
GROUP BY Doctor_ID;
```

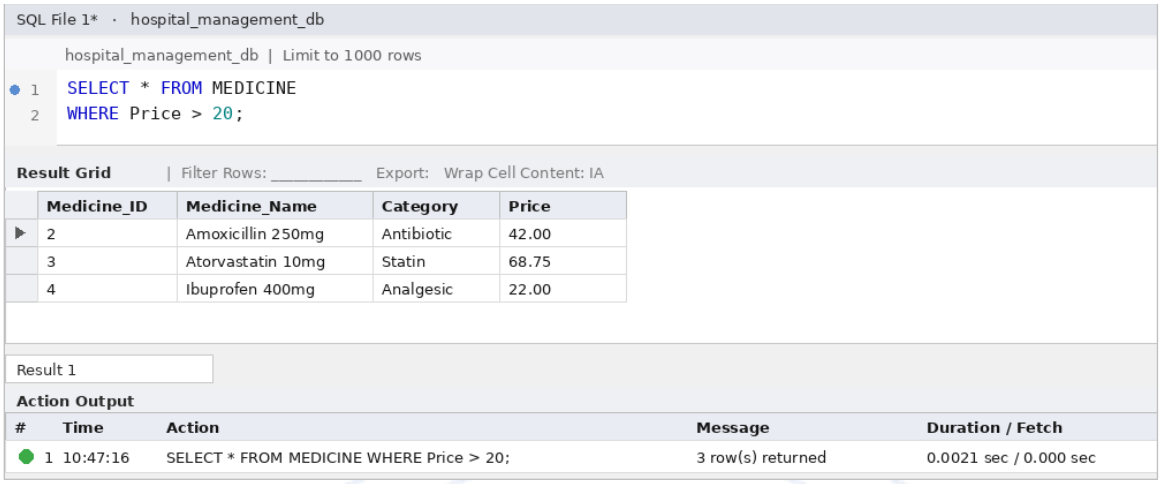
Output:



Q18. Medicines priced above 20

```
SELECT * FROM MEDICINE
WHERE Price > 20;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT * FROM MEDICINE
2  WHERE Price > 20;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Medicine_ID	Medicine_Name	Category	Price
2	Amoxicillin 250mg	Antibiotic	42.00
3	Atorvastatin 10mg	Statin	68.75
4	Ibuprofen 400mg	Analgesic	22.00

Result 1

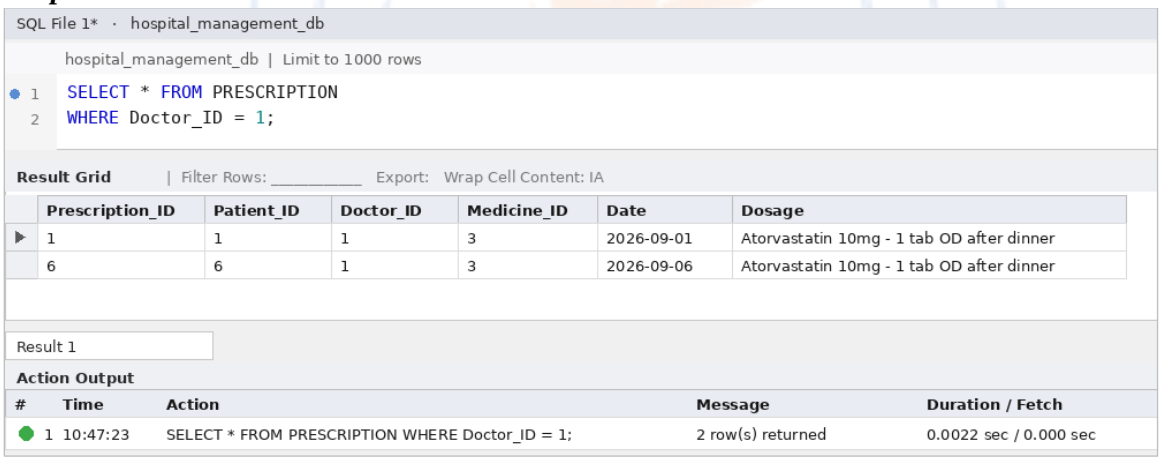
Action Output

#	Time	Action	Message	Duration / Fetch
1	10:47:16	SELECT * FROM MEDICINE WHERE Price > 20;	3 row(s) returned	0.0021 sec / 0.000 sec

Q19. Prescriptions issued by a specific doctor

```
SELECT * FROM PRESCRIPTION
WHERE Doctor_ID = 1;
```

Output:



SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT * FROM PRESCRIPTION
2  WHERE Doctor_ID = 1;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Prescription_ID	Patient_ID	Doctor_ID	Medicine_ID	Date	Dosage
1	1	1	3	2026-09-01	Atorvastatin 10mg - 1 tab OD after dinner
6	6	1	3	2026-09-06	Atorvastatin 10mg - 1 tab OD after dinner

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:47:23	SELECT * FROM PRESCRIPTION WHERE Doctor_ID = 1;	2 row(s) returned	0.0022 sec / 0.000 sec

Q20. Pending bills sorted by amount (descending)

```
SELECT Bill_ID, Patient_ID, Amount
FROM BILLING
WHERE Payment_Status = 'Pending'
ORDER BY Amount DESC;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Bill_ID, Patient_ID, Amount
2  FROM BILLING
3  WHERE Payment_Status = 'Pending'
4  ORDER BY Amount DESC;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Bill_ID	Patient_ID	Amount
3	3	8000.00
6	6	1200.00
5	5	500.00

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:48:30	SELECT Bill_ID, Patient_ID, Amount FROM BILLING WHERE Paymen...	3 row(s) returned	0.0023 sec / 0.000 sec

Q21. Number of doctors in each department

```
SELECT Department_ID, COUNT(*) AS Doctor_Count
FROM DOCTOR
GROUP BY Department_ID;
```

Output:

SQL File 1* · hospital_management_db

hospital_management_db | Limit to 1000 rows

```
1  SELECT Department_ID, COUNT(*) AS Doctor_Count
2  FROM DOCTOR
3  GROUP BY Department_ID;
```

Result Grid | Filter Rows: _____ Export: Wrap Cell Content: IA

Department_ID	Doctor_Count
1	1
2	1
3	1
4	1
5	1

Result 1

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:48:37	SELECT Department_ID, COUNT(*) AS Doctor_Count FROM DOCTOR G...	5 row(s) returned	0.0024 sec / 0.000 sec

10. CONCLUSION

The Hospital Management Database System provides a structured relational database solution for managing core hospital information. The design organizes departments, patients, doctors, rooms, appointments, admissions, treatments, prescriptions, medicines, and billing into ten related tables, and applies Primary Keys, Foreign Keys, uniqueness rules, NOT NULL conditions, CHECK constraints, and appropriate ON UPDATE / ON DELETE actions to support data integrity. The normalization process up to 3NF reduces redundancy and update anomalies. The SQL implementation, sample data, and query set — covering CRUD operations, joins, aggregation, a correlated subquery, and a view — demonstrate how the database supports insertion, modification, retrieval, and hospital reporting. The project therefore provides a suitable DBMS cornerstone implementation that can be extended into a complete hospital information system.

11. FUTURE ENHANCEMENTS

- Role-based access for administrators, doctors, and other authorized hospital staff.
- Staff and nursing-duty management module.
- Insurance and cashless-claim processing linked to Billing.
- Laboratory test records and diagnostic report management.
- Pharmacy inventory and stock management linked to Medicine.
- Automated report and analytics dashboards for occupancy, revenue, and doctor workload.
- Backup, recovery, audit logging, and stronger database security for patient data.

REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.

2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

3. MySQL 8.0 Reference Manual, Oracle Corporation, https://dev.mysql.com/doc/